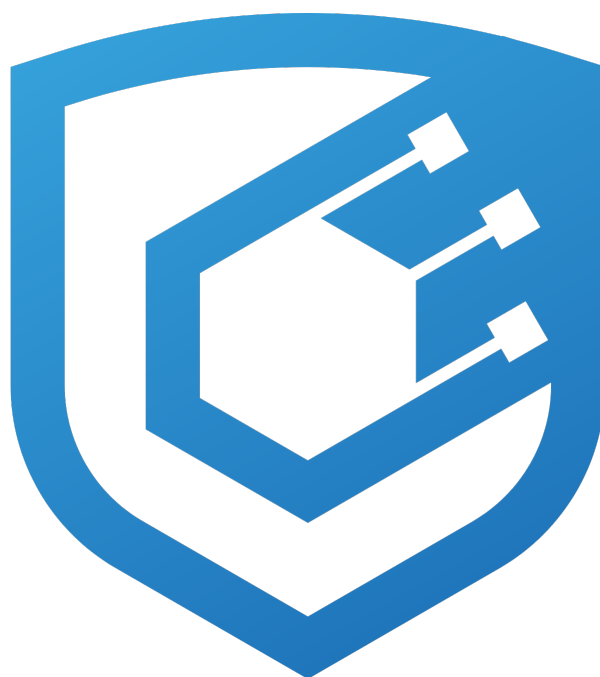




Linea Forced Transactions Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[Stalin](#)

June 18, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
7	Findings	8
7.1	Low Risk	8
7.1.1	Inconsistent <code>blockNumberDeadline</code> for forced transactions can lead to DoS newer forced transactions requests	8
7.2	Informational	10
7.2.1	Use named mappings to explicitly denote the purpose of keys and values	10
7.2.2	Missing code to track and withdraw forced transaction fees	10
7.2.3	Remove TODO comments	10
7.2.4	Inline call to <code>ForcedTransactionGateway::_buildAccessList</code> since variable <code>accessList</code> used only once in <code>submitForcedTransaction</code>	11
7.2.5	In Solidity don't initialize to default values	11
7.2.6	Incorrect natspec above <code>LineaRollupBase::_computePublicInput</code>	11
7.3	Gas Optimization	16
7.3.1	Remove unused storage slots in new contracts	16
7.3.2	More efficient usage of <code>LibRLP</code> in <code>ForcedTransactionGateway::_buildAccessList</code> , <code>submitForcedTransaction</code>	16
7.3.3	Refactor to avoid two calls to <code>AddressFilter::addressIsFiltered</code>	16

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Linea is a [Type 2](#) zkEVM L2 rollup with full EVM equivalence which aims to provide the "Ethereum experience" at scale and has been previously audited by Cyfrin.

The focus of this audit was the new [Forced Transaction](#) functionality which aims to provide a "processing guarantee" allowing users to submit L1 transactions which must be processed by the L2 sequencer within a specified block deadline.

5 Audit Scope

The audit scope was limited to Forced Transaction changes in [PR897](#):

```
// see smart contract related changes in
// https://github.com/Consensys/linea-monorepo/pull/897/changes?mode=virtualization

// new files
contracts/src/libraries/FinalizedStateHashing.sol
contracts/src/rollup/forcedTransactions/AddressFilter.sol
contracts/src/rollup/forcedTransactions/ForcedTransactionGateway.sol

// only `reinitializeLineaRollupV9`
contracts/src/rollup/LineaRollup.sol

// new stuff related to forced transactions eg `FORCED_TRANSACTION_SENDER_ROLE`,
// `nextForcedTransactionNumber`, `forcedTransactionL2BlockNumbers`, `forcedTransactionRollingHashes`,
// `getRequiredForcedTransactionFields`, `storeForcedTransaction` plus changes to existing functions
contracts/src/rollup/LineaRollupBase.sol

// small change related to `lastFinalizedForcedTransaction`
contracts/src/rollup/LivenessRecovery.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Linea Forced Transactions](#) smart contracts provided by [Linea](#). In this period, a total of 10 issues were found.

The findings consist of 1 Low severity issue with the remainder being informational and gas optimizations.

Deployment Bytecode Verification 1

At the conclusion of the audit after all mitigations were successfully concluded, Cyfrin was asked to verify that at commit [70dde7](#) the locally compiled bytecode matched the [expected bytecode](#) listed in that commit hash. We used the following reproducible process to do this.

1. Prepare the repo:

```
gh repo clone Consensys/linea-monorepo
cd linea-monorepo
git checkout 70dde763052985302cfaf4ea8e98010d11278ea1
pnpm install --ignore-scripts // to get around an error related to better-sqlite3
cd node_modules/.pnpm/c-kzg@4.1.0/node_modules/c-kzg && npx node-gyp rebuild // rebuild c-kzg native
↳ bindings
cd contracts
npx hardhat test
```

2. Diff the locally compiled bytecode against the [expected](#):

```
linea-monorepo/contracts$ diff contracts/build/src/rollup/forcedTransactions/ForcedTransactionGateway.s |
↳ ol/ForcedTransactionGateway.json
↳ contracts/deployments/bytecode/2026-02-20/ForcedTransactionGateway.json

linea-monorepo/contracts$ diff
↳ contracts/build/src/rollup/forcedTransactions/AddressFilter.sol/AddressFilter.json
↳ contracts/deployments/bytecode/2026-02-20/AddressFilter.json

linea-monorepo/contracts$ diff contracts/build/src/rollup/LineaRollup.sol/LineaRollup.json
↳ contracts/deployments/bytecode/2026-02-20/LineaRollup.json

linea-monorepo/contracts$ diff contracts/build/src/rollup/Validium.sol/Validium.json
↳ contracts/deployments/bytecode/2026-02-20/Validium.json
```

3. Verify the immutableReferences of the ForcedTransactionGateway:

- Locate the generated hardhat dbg file for the locally compiled contracts and extract the immutableReferences from the referenced file under the build-info folder.

Deployment Bytecode Verification 2

After the forced transactions code was merged to main at commit [4ddbaba](#), Cyfrin was asked to verify that the build output in `linea-monorepo/contracts/deployments/bytecode/2026-02-20` still matches the locally compiled code (allowing for the additional `immutableReferences` block which is appended to the deployments JSONs after the hardhat compile).

1. Prepare the repo:

```
gh repo clone Consensys/linea-monorepo
cd linea-monorepo
git checkout 4ddbaba806f1a31719539cd507d7bde29e4b481d
pnpm install --ignore-scripts // to get around an error related to better-sqlite3
cd node_modules/.pnpm/c-kzg@4.1.0/node_modules/c-kzg && npx node-gyp rebuild // rebuild c-kzg native
↳ bindings
cd contracts
npx hardhat test
```

2. Diff the locally compiled bytecode against the [expected](#):

```
linea-monorepo/contracts$ diff build/src/rollup/forcedTransactions/AddressFilter.sol/AddressFilter.json
↳ deployments/bytecode/2026-02-20/AddressFilter.json

linea-monorepo/contracts$ diff build/src/rollup/LineaRollup.sol/LineaRollup.json
↳ deployments/bytecode/2026-02-20/LineaRollup.json

linea-monorepo/contracts$ diff build/src/rollup/Validium.sol/Validium.json
↳ deployments/bytecode/2026-02-20/Validium.json
```

AddressFilter, LineaRollup and Validium are byte-identical between the locally compiled artifacts and the expected artifacts in deployments/bytecode/2026-02-20.

3. For ForcedTransactionGateway the only difference between the locally compiled artifact and the expected artifact is the additional immutableReferences block appended to the deployments JSON:

```
linea-monorepo/contracts$ diff
↳ build/src/rollup/forcedTransactions/ForcedTransactionGateway.sol/ForcedTransactionGateway.json
↳ deployments/bytecode/2026-02-20/ForcedTransactionGateway.json
```

4. Verify the immutableReferences of the ForcedTransactionGateway:

- Locate the generated hardhat dbg file build/src/rollup/forcedTransactions/ForcedTransactionGateway.sol/ForcedTransactionGateway.dbg for the locally compiled contract; it points at the corresponding build-info file.
- Extract the immutableReferences for ForcedTransactionGateway from that build-info file:

```
jq '.output.contracts["src/rollup/forcedTransactions/ForcedTransactionGateway.sol"].ForcedTransactionGateway.
↳ eway.evm.deployedBytecode.immutableReferences' build/build-info/<hash>.json
```

The locally compiled immutableReferences use different top-level AST identifiers than those embedded in the expected deployments JSON (AST IDs depend on the exact set of source files passed to the compiler), but the structure — the number of immutables, and the length and start offsets of every entry — is identical:

Local AST ID	Deployments AST ID	Entries (length @ start)
33772	33169	32 @ 1031, 32 @ 2307, 32 @ 3796
33775	33172	32 @ 610, 32 @ 2712
33778	33175	32 @ 1133, 32 @ 3571
33781	33178	32 @ 949, 32 @ 3604
33784	33181	32 @ 1082, 32 @ 1795
33787	33184	32 @ 750, 32 @ 1889
33790	33187	32 @ 338, 32 @ 3672
33794	33191	32 @ 455, 32 @ 3117, 32 @ 3406

Every immutable placeholder slot in the locally compiled deployed bytecode coincides exactly with an immutable placeholder slot in the expected deployed bytecode in deployments/bytecode/2026-02-20, confirming that the build output in that folder still matches the compiled code at commit [4ddbaba](#).

Deployment Bytecode Verification 3

While deploying to their Devnet, Consensys discovered the LineaRollup deployed bytecode exceeded the EIP-170 contract size limit and recompiled it with the optimizer runs setting lowered from 10_000 to 200 (overrides apply only to src/rollup/LineaRollup.sol; AddressFilter, ForcedTransactionGateway and Validium remain

at runs: 10_000). Cyfrin was asked to verify that at commit [6d0c5d4](#) the locally compiled bytecode still matches the [expected bytecode](#) listed in that commit hash.

1. Prepare the repo:

```
gh repo clone Consensys/linea-monorepo
cd linea-monorepo
git checkout 6d0c5d481a7b0e2a1e5ad829890f4cab8032cb66
pnpm install --ignore-scripts // to get around an error related to better-sqlite3
cd node_modules/.pnpm/c-kzg@4.1.0/node_modules/c-kzg && npx node-gyp rebuild // rebuild c-kzg native
↪ bindings
cd contracts
npx hardhat test
```

2. Diff the locally compiled bytecode against the [expected](#):

```
linea-monorepo/contracts$ diff build/src/rollup/forcedTransactions/AddressFilter.sol/AddressFilter.json
↪ deployments/bytecode/2026-02-20/AddressFilter.json

linea-monorepo/contracts$ diff build/src/rollup/LineaRollup.sol/LineaRollup.json
↪ deployments/bytecode/2026-02-20/LineaRollup.json

linea-monorepo/contracts$ diff build/src/rollup/Validium.sol/Validium.json
↪ deployments/bytecode/2026-02-20/Validium.json
```

AddressFilter, LineaRollup and Validium are byte-identical between the locally compiled artifacts and the expected artifacts in deployments/bytecode/2026-02-20.

3. For ForcedTransactionGateway the only difference between the locally compiled artifact and the expected artifact is the additional immutableReferences block appended to the deployments JSON:

```
linea-monorepo/contracts$ diff
↪ build/src/rollup/forcedTransactions/ForcedTransactionGateway.sol/ForcedTransactionGateway.json
↪ deployments/bytecode/2026-02-20/ForcedTransactionGateway.json
```

4. Verify the immutableReferences of the ForcedTransactionGateway:

- Locate the generated hardhat dbg file build/src/rollup/forcedTransactions/ForcedTransactionGateway.sol/ForcedTransactionGateway.dbg for the locally compiled contract; it points at the corresponding build-info file.
- Extract the immutableReferences for ForcedTransactionGateway from that build-info file:

```
jq '.output.contracts."src/rollup/forcedTransactions/ForcedTransactionGateway.sol".ForcedTransactionGateway |
↪ away.evm.deployedBytecode.immutableReferences' build/build-info/<hash>.json
```

As in Verification 2, the locally compiled immutableReferences use different top-level AST identifiers than those embedded in the expected deployments JSON (AST IDs depend on the exact set of source files passed to the compiler), but the structure — the number of immutables, and the length and start offsets of every entry — is identical:

Local AST ID	Deployments AST ID	Entries (length @ start)
33772	33169	32 @ 1031, 32 @ 2307, 32 @ 3796
33775	33172	32 @ 610, 32 @ 2712
33778	33175	32 @ 1133, 32 @ 3571
33781	33178	32 @ 949, 32 @ 3604
33784	33181	32 @ 1082, 32 @ 1795
33787	33184	32 @ 750, 32 @ 1889

Local AST ID	Deployments AST ID	Entries (length @ start)
33790	33187	32 @ 338, 32 @ 3672
33794	33191	32 @ 455, 32 @ 3117, 32 @ 3406

Every immutable placeholder slot in the locally compiled deployed bytecode coincides exactly with an immutable placeholder slot in the expected deployed bytecode in `deployments/bytecode/2026-02-20`, confirming that — after the `LineaRollup optimizer-runs` change — the build output in that folder still matches the compiled code at commit [6d0c5d4](#).

Deployment Bytecode Verification 4

In May 2026 the `linea-monorepo` repository was migrated to the Linux Foundation Decentralized Trust (LFDT) and renamed; it now lives at [LFDT-Lineth/lineth-monorepo](#). Cyfrin confirmed the commit history referenced in Verifications 1-3 remains intact in the new repository — commits [70dde7](#), [4ddbeba](#) and [6d0c5d4](#) are all present with their original contents - and that the expected artifacts in `contracts/deployments/bytecode/2026-02-20` are byte-identical between commit [6d0c5d4](#) and the current `main` branch.

After deploying to Ethereum mainnet, Cyfrin was asked to verify that the on-chain runtime bytecode of the deployed `AddressFilter` and `LineaRollup` implementation contracts matches the expected bytecode stored in `contracts/deployments/bytecode/2026-02-20`:

Contract	Deployed Address
<code>AddressFilter</code>	0x526AE78F0103Ae73F05449ae30eb626C1003784E
<code>LineaRollup (impl)</code>	0x59290394dDC1cF84e671701A929710643c343530

1. Fetch the on-chain runtime bytecode from Ethereum mainnet (verified at block 25344318):

```
cast code 0x526AE78F0103Ae73F05449ae30eb626C1003784E --rpc-url <mainnet-rpc> > AddressFilter.onchain
cast code 0x59290394dDC1cF84e671701A929710643c343530 --rpc-url <mainnet-rpc> > LineaRollup.onchain
```

2. Compare against the `deployedBytecode` field of the expected artifacts:

```
diff <(jq -r .deployedBytecode AddressFilter.json) AddressFilter.onchain
diff <(jq -r .deployedBytecode LineaRollup.json) LineaRollup.onchain
```

The on-chain runtime bytecode of both contracts is byte-identical to the `deployedBytecode` of the expected artifacts, including the trailing metadata (CBOR) hash. This confirms the deployed contracts were compiled from the same sources with the same compiler version (0.8.33) and settings (EVM version `Osaka`; optimizer runs = 10_000 for `AddressFilter`, 200 for `LineaRollup`) recorded in `contracts/deployments/bytecode/2026-02-20`:

Contract	keccak256(on-chain runtime bytecode)	Result
<code>AddressFilter</code>	<code>0x9802e00fbcfe819915bec75ceb50f3ca1534fcef5a01ab7fc49109a217d7fb2e</code>	byte-identical
<code>LineaRollup</code>	<code>0xf4830e7ba3afea4b3fb94af86fb63db9e5200ad1c7c9722c0b18a4d26fb7b58f</code>	byte-identical

Summary

Project Name	Linea Forced Transactions
Repository	linea-monorepo
Commit	990f688e9576...
Fix Commit	053093da2e7a...
Audit Timeline	Feb 2nd - Feb 6th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Informational	6
Gas Optimizations	3
Total Issues	10

Summary of Findings

[L-1] Inconsistent <code>blockNumberDeadline</code> for forced transactions can lead to DoS newer forced transactions requests	Resolved
[I-1] Use named mappings to explicitly denote the purpose of keys and values	Resolved
[I-2] Missing code to track and withdraw forced transaction fees	Acknowledged
[I-3] Remove TODO comments	Resolved
[I-4] Inline call to <code>ForcedTransactionGateway::_buildAccessList</code> since variable <code>accessList</code> used only once in <code>submitForcedTransaction</code>	Resolved
[I-5] In Solidity don't initialize to default values	Resolved
[I-6] Incorrect natspec above <code>LineaRollupBase::_computePublicInput</code>	Resolved
[G-1] Remove unused storage slots in new contracts	Resolved
[G-2] More efficient usage of <code>LibRLP</code> in <code>ForcedTransactionGateway::_buildAccessList</code> , <code>submitForcedTransaction</code>	Resolved
[G-3] Refactor to avoid two calls to <code>AddressFilter::addressIsFiltered</code>	Resolved

7 Findings

7.1 Low Risk

7.1.1 Inconsistent blockNumberDeadline for forced transactions can lead to DoS newer forced transactions requests

Description: When submitting a forced transaction on L1, the contract calculates blockNumberDeadline using the next formula:

```
uint256 blockNumberDeadline;
unchecked {
    /// @dev The computation uses 1s block time making block number and seconds interchangeable,
    /// while the chain might currently differ at >1s, this gives additional inclusion time.
    blockNumberDeadline =
        currentFinalizedL2BlockNumber +
        block.timestamp -
        _lastFinalizedState.timestamp +
        L2_BLOCK_BUFFER;
}
```

The problem is that the delta between block.timestamp & _lastFinalizedState.timestamp varies significantly depending on when the forced transaction is submitted relative to the most recent finalization. This creates inconsistent deadlines for users:

- Just before a new block is finalized - **Longer deadline:** _lastFinalizedState.timestamp is very close to block.timestamp → small delta → higher blockNumberDeadline (longer deadline)
- Just after a new block was finalized - **Shorter deadline:** _lastFinalizedState.timestamp is significantly older than block.timestamp → large delta → lower blockNumberDeadline (shorter deadline)

This means two users submitting a forced transaction at almost the same real-world time can receive very different deadlines, purely based on finalization timing luck.

The root cause of the problem stems from the fact that the formula is meant to treat block numbers and seconds interchangeably, which means, as if the block's production rate would be 1 per second, but this is not the case in [Linea](#), the current block rate is ~1 block every 2 seconds.

Impact: Deadline varies significantly depending on how recently a block was finalized:

- Forced tx sent right after finalization → shorter deadline
- Forced tx sent right before next finalization → longer deadline

Proof of Concept: The next example demonstrates the discrepancy in the deadlines. Assume the blocks are finalized every 10k seconds at a rate of 1 block every 2 seconds. (That means, during each finalization, 5k blocks will be finalized).

Current state on the rollup is as:

- currentFinalizedL2BlockNumber = 50_000
- _lastFinalizedState.timestamp = 90_000

The next finalization would update values to:

- currentFinalizedL2BlockNumber = 55_000
- _lastFinalizedState.timestamp = 100_000

Here is what happens to the deadline when a forced tx is submitted right before the next finalization occurs:

forced tx	Approx. real time	Time since last finalized state	Finalized L2 block #	blockNumberDeadline
1	t 99,999 s	~9,999 s	50,000	59,999

forced tx	Approx. real time	Time since last finalized state	Finalized L2 block #	blockNumberDeadline
2	t 100,001 s	~1 s	55,000	55,001

This discrepancy can lead to a DoS from subsequent forced transaction requests, as the newest transaction would have a lower blockNumberDeadline than the last stored forced transaction; Rollup::storeForcedTransaction explicitly reverts execution.

```
function storeForcedTransaction(
    ...
) external payable virtual onlyRole(FORCED_TRANSACTION_SENDER_ROLE) {
    unchecked {
        ...

        uint256 forcedTransactionNumber = nextForcedTransactionNumber++;

        //@audit-info => deadline for previous forced tx must be beyond the block number deadline assigned
        ↳ for the newest forced transaction
        require(
            forcedTransactionL2BlockNumbers[forcedTransactionNumber - 1] < _blockNumberDeadline,
            ForcedTransactionExistsForBlockOrIsTooLow(_blockNumberDeadline)
        );

        forcedTransactionRollingHashes[forcedTransactionNumber] = _forcedTransactionRollingHash;
        forcedTransactionL2BlockNumbers[forcedTransactionNumber] = _blockNumberDeadline;

        ...
    }
}
```

The table below shows the outcome of requesting forced transactions using the original example of this issue: | Tx | Submitted blockNumberDeadline | Assigned forcedTransactionNumber | Previous deadline (index n-1) | Check (prev < new) | Outcome |

1	59,999	1	0 (default/not set)	0 < 59,999	→ yes Success
2	55,001	2	59,999	59,999 < 55,001	→ no Revert

Recommended Mitigation: Consider refactoring the blockNumberDeadline formula to account for the actual L2 block rate.

Linea: Fixed in commit [8d9b0f](#) && [05309](#).

Cyfrin: Verified. The formula for calculating blockNumberDeadline now adjusts the elapsed time for the finalization period based on the L2 block rate. Now, the blockNumberDeadline is adjusted in the ForcedTransactionGateway to add a buffer in the edge case where the calculated blockNumberDeadline is lower than the last recorded deadline on the Rollup; This buffer mitigates the case when the L2 block rate production deviates from the L2_BLOCK_DURATION_SECONDS.

7.2 Informational

7.2.1 Use named mappings to explicitly denote the purpose of keys and values

Description: Use named mappings to explicitly denote the purpose of keys and values:

```
AddressFilter.sol
15: mapping(address => bool) internal filteredAddresses;
```

Linea: Fixed in commit [857c4b7](#).

Cyfrin: Verified.

7.2.2 Missing code to track and withdraw forced transaction fees

Description: When users call `ForcedTransactionGateway::submitForcedTransaction` they send the required fee as `msg.value`. This function passes the fee (`msg.value`) to `LineaRollupBase::storeForcedTransaction` but `LineaRollupBase` doesn't:

- track the total amount of received forced transaction fees
- contain a function to withdraw the fees

Consider at least tracking the total amount of received forced transaction fees and potentially adding a function to withdraw them.

Linea: Acknowledged; adding a withdraw function by Linea for any value (tracked or untracked) introduces questions and undue suspicion. Creating a function for this would cause more community concern than needed, so this is not really an option.

Any fees paid are ok to be "donated" to the ecosystem and provide extra cushioning or staking value. We don't expect this to be used regularly and the amounts are mostly negligible.

The amounts can be calculated easily by using the `ForcedTransactionAdded` type events and getting the transaction value sent as this is a 1:1 for the fees paid - no more, no less.

If there was ever a withdraw of this specific amount, it would go through a security council upgrade with a fixed call with the amount that can be publicly traceable.

7.2.3 Remove TODO comments

Description: Remove TODO comments:

```
rollup/LineaRollupBase.sol
124: // TODO check the layout of these variables
```

Comparing this audit's `LineaRollup` storage layout to the previous "mixed upgrade" version using `forge inspect -R "@openzeppelin/=contracts/node_modules/@openzeppelin/" --hardhat --evm-version cancun` `LineaRollup` `storageLayout` shows that the new storage slots are appended after the previously existing `shnarfProvider` slot which is correct:

Name	Type	Slot	Offset	Bytes
# identical				
shnarfProvider	contract IProvideShnarf	449	0	20
# new, over-write previous gap				
nextForcedTransactionNumber	uint256	450	0	32
forcedTransactionL2BlockNumbers	mapping(uint256 => uint256)	451	0	32
forcedTransactionRollingHashes	mapping(uint256 => bytes32)	452	0	32
forcedTransactionFeeInWei	uint256	453	0	32
addressFilter	contract IAddressFilter	454	0	20
# pushed down 5 slots				

__gap_LinearRollup	uint256[50]	455	0	1600
__gap_LivenessRecoveryOperator	uint256[50]	505	0	1600

The storage gap LinearRollupBase::__gap_LinearRollup has not been reduced but this appears to be safe since it is still at the end of the storage layout.

Linea: Fixed in commit [857c4b7](#).

Cyfrin: Verified.

7.2.4 Inline call to ForcedTransactionGateway::_buildAccessList since variable accessList used only once in submitForcedTransaction

Description: Inline call to ForcedTransactionGateway::_buildAccessList since variable accessList used only once in submitForcedTransaction:

```
- LibRLP.List memory accessList = _buildAccessList(_forcedTransaction.accessList);
LibRLP.List memory transactionFieldList = LibRLP.p();
transactionFieldList = LibRLP.p(transactionFieldList, DESTINATION_CHAIN_ID);
transactionFieldList = LibRLP.p(transactionFieldList, _forcedTransaction.nonce);
transactionFieldList = LibRLP.p(transactionFieldList, _forcedTransaction.maxPriorityFeePerGas);
transactionFieldList = LibRLP.p(transactionFieldList, _forcedTransaction.maxFeePerGas);
transactionFieldList = LibRLP.p(transactionFieldList, _forcedTransaction.gasLimit);

if (_forcedTransaction.to == address(0)) {
    transactionFieldList = LibRLP.p(transactionFieldList, bytes(""));
} else {
    transactionFieldList = LibRLP.p(transactionFieldList, _forcedTransaction.to);
}
transactionFieldList = LibRLP.p(transactionFieldList, _forcedTransaction.value);
transactionFieldList = LibRLP.p(transactionFieldList, _forcedTransaction.input);
- transactionFieldList = LibRLP.p(transactionFieldList, accessList);
+ transactionFieldList = LibRLP.p(transactionFieldList,
  ↳ _buildAccessList(_forcedTransaction.accessList));
```

Linea: Fixed in commit [857c4b7](#).

Cyfrin: Verified.

7.2.5 In Solidity don't initialize to default values

Description: In Solidity don't initialize to default values:

```
rollup/LinearRollupBase.sol
534:     for (uint256 i = 0; i < _filteredAddresses.length; i++) {
```

Linea: Fixed in commit [857c4b7](#).

Cyfrin: Verified.

7.2.6 Incorrect natspec above LinearRollupBase::_computePublicInput

Description: Incorrect natspec above LinearRollupBase::_computePublicInput:

```
* 0x220    l2MerkleRootsLengthLocation
- * 0x240    l2MessagingBlocksOffsetsLengthLocation
+ * 0x240    filteredAddressesLengthLocation
+ * 0x260    l2MessagingBlocksOffsetsLengthLocation
```

Also consider changing the name convention from "...LengthLocation" to "...OffsetPointer". The values stored at those offsets are not length locations, they're offset pointers. The offset pointer indicates where in the tail to find the length-prefixed array data:

- `calldataload(add(_finalizationData, 0x240))` returns a relative byte offset (e.g., 0x2a0)
- adding that offset to `_finalizationData` gives the length location: `add(_finalizationData, 0x2a0) →` where the array length lives
- the actual array elements start at `add(_finalizationData, 0x2c0)`

So they're offset pointers that resolve to length locations, but they aren't the length locations themselves.

Proof of Concept: The Foundry tests are broken; to get the PoC working first remove old evm versions from `foundry.toml`:

```
[profile.default]
src = 'src'
out = 'out'
libs = ['node_modules', 'lib']
cache_path = 'cache_forge'
foundry_version = "stable"

# Test settings
match-path = 'test/foundry/*'

# Default solc compiler settings
evm_version = "osaka"
optimizer = true
optimizer_runs = 10_000
```

Then comment out the code in `contracts/test/foundry/LineaRollup.t.sol` since it no longer compiles.

Afterwards add new PoC file `contracts/test/foundry/CalldataLayoutProof.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.33;

import "forge-std/Test.sol";

/**
 * @title Proof that FinalizationDataV4 NatSpec comments have wrong calldata offsets.
 * @dev The NatSpec in _computePublicInput claims:
 *      0x220    l2MerkleRootsLengthLocation
 *      0x240    l2MessagingBlocksOffsetsLengthLocation
 *
 * But the actual ABI-encoded layout is:
 *      0x220    l2MerkleRoots offset pointer
 *      0x240    filteredAddresses offset pointer      ← MISSING from NatSpec
 *      0x260    l2MessagingBlocksOffsets offset pointer ← NatSpec says 0x240
 *
 * This test proves it by encoding a FinalizationDataV4 struct and inspecting
 * the raw calldata bytes at each offset.
 */

// Minimal reproduction of the struct types
struct ShnarfData {
    bytes32 parentShnarf;
    bytes32 snarkHash;
    bytes32 finalStateRootHash;
    bytes32 dataEvaluationPoint;
    bytes32 dataEvaluationClaim;
}

struct FinalizationDataV4 {
    bytes32 parentStateRootHash;           // 0x00
    uint256 endBlockNumber;                // 0x20
    ShnarfData shnarfData;                 // 0x40-0xc0 (5 slots inline)
```

```

uint256 lastFinalizedTimestamp;           // 0xe0
uint256 finalTimestamp;                   // 0x100
bytes32 lastFinalizedL1RollingHash;       // 0x120
bytes32 l1RollingHash;                    // 0x140
uint256 lastFinalizedL1RollingHashMessageNumber; // 0x160
uint256 l1RollingHashMessageNumber;      // 0x180
uint256 l2MerkleTreesDepth;               // 0x1a0
uint256 lastFinalizedForcedTransactionNumber; // 0x1c0
uint256 finalForcedTransactionNumber;     // 0x1e0
bytes32 lastFinalizedForcedTransactionRollingHash; // 0x200
bytes32[] l2MerkleRoots;                   // 0x220 offset pointer
address[] filteredAddresses;               // 0x240 offset pointer
bytes l2MessagingBlocksOffsets;           // 0x260 offset pointer
}

contract CalldataInspector {
    /// @dev We use calldata to get the exact ABI-encoded layout that
    ///       LineaRollupBase._computePublicInput operates on.
    ///
    ///       Returns the raw 32-byte words at offsets 0x220, 0x240, 0x260
    ///       within the struct's calldata encoding.
    function inspect(
        FinalizationDataV4 calldata _data
    )
        external
        pure
        returns (
            uint256 wordAt0x220,
            uint256 wordAt0x240,
            uint256 wordAt0x260,
            // Also return the actual dynamic data to prove the offset pointers
            // resolve to the correct arrays
            uint256 merkleRootsLength,
            bytes32 merkleRootsFirstElement,
            uint256 filteredAddressesLength,
            address filteredAddressesFirstElement,
            uint256 messagingOffsetsLength
        )
    {
        assembly {
            // Read the raw 32-byte words at each offset relative to _data
            wordAt0x220 := calldataload(add(_data, 0x220))
            wordAt0x240 := calldataload(add(_data, 0x240))
            wordAt0x260 := calldataload(add(_data, 0x260))

            // Dereference offset at 0x220 → should point to l2MerkleRoots
            let merkleRootsLoc := add(_data, wordAt0x220)
            merkleRootsLength := calldataload(merkleRootsLoc)
            merkleRootsFirstElement := calldataload(add(merkleRootsLoc, 0x20))

            // Dereference offset at 0x240 → should point to filteredAddresses
            let filteredLoc := add(_data, wordAt0x240)
            filteredAddressesLength := calldataload(filteredLoc)
            filteredAddressesFirstElement := calldataload(add(filteredLoc, 0x20))

            // Dereference offset at 0x260 → should point to l2MessagingBlocksOffsets
            let messagingLoc := add(_data, wordAt0x260)
            messagingOffsetsLength := calldataload(messagingLoc)
        }
    }
}

contract CalldataLayoutProofTest is Test {

```

```

CalldataInspector inspector;

function setUp() public {
    inspector = new CalldataInspector();
}

function test_finalizationDataV4_calldata_layout() public view {
    // Build a FinalizationDataV4 with recognizable sentinel values
    // in the dynamic arrays so we can verify correct dereferencing.

    bytes32[] memory merkleRoots = new bytes32[](2);
    merkleRoots[0] = bytes32(uint256(0xAAAA));
    merkleRoots[1] = bytes32(uint256(0xBBBB));

    address[] memory filtered = new address[](1);
    filtered[0] = address(0xDEAD);

    bytes memory messagingOffsets = hex"CCDD";

    FinalizationDataV4 memory data = FinalizationDataV4({
        parentStateRootHash: bytes32(uint256(1)),
        endBlockNumber: 1000,
        shnarfData: ShnarfData({
            parentShnarf: bytes32(uint256(2)),
            snarkHash: bytes32(uint256(3)),
            finalStateRootHash: bytes32(uint256(4)),
            dataEvaluationPoint: bytes32(uint256(5)),
            dataEvaluationClaim: bytes32(uint256(6))
        }),
        lastFinalizedTimestamp: 7,
        finalTimestamp: 8,
        lastFinalizedL1RollingHash: bytes32(uint256(9)),
        l1RollingHash: bytes32(uint256(10)),
        lastFinalizedL1RollingHashMessageNumber: 11,
        l1RollingHashMessageNumber: 12,
        l2MerkleTreesDepth: 5,
        lastFinalizedForcedTransactionNumber: 13,
        finalForcedTransactionNumber: 14,
        lastFinalizedForcedTransactionRollingHash: bytes32(uint256(15)),
        l2MerkleRoots: merkleRoots,
        filteredAddresses: filtered,
        l2MessagingBlocksOffsets: messagingOffsets
    });

    (
        uint256 wordAt0x220,
        uint256 wordAt0x240,
        uint256 wordAt0x260,
        uint256 merkleRootsLength,
        bytes32 merkleRootsFirstElement,
        uint256 filteredAddressesLength,
        address filteredAddressesFirstElement,
        uint256 messagingOffsetsLength
    ) = inspector.inspect(data);

    //
    // PROOF 1: All three offsets at 0x220, 0x240, 0x260 are DISTINCT
    //           offset pointers (not zero, not the same value).
    //
    //           If the NatSpec were correct (only 2 dynamic fields with
    //           offsets at 0x220 and 0x240), then 0x260 would be the
    //           start of the tail data, not an offset pointer.
    //

```

```

assertTrue(wordAt0x220 != wordAt0x240, "0x220 and 0x240 should be different offset pointers");
assertTrue(wordAt0x240 != wordAt0x260, "0x240 and 0x260 should be different offset pointers");
assertTrue(wordAt0x220 != wordAt0x260, "0x220 and 0x260 should be different offset pointers");

// Offsets should be strictly increasing (arrays laid out in order in tail)
assertTrue(wordAt0x220 < wordAt0x240, "l2MerkleRoots offset < filteredAddresses offset");
assertTrue(wordAt0x240 < wordAt0x260, "filteredAddresses offset < l2MessagingBlocksOffsets
↳ offset");

//
// PROOF 2: Dereferencing offset at 0x220 yields l2MerkleRoots
//

assertEq(merkleRootsLength, 2, "0x220 -> l2MerkleRoots: length should be 2");
assertEq(merkleRootsFirstElement, bytes32(uint256(0xAAAA)), "0x220 -> l2MerkleRoots[0] should be
↳ 0xAAAA");

//
// PROOF 3: Dereferencing offset at 0x240 yields filteredAddresses
// THIS IS THE FIELD MISSING FROM THE NATSPEC COMMENT.
// The NatSpec claims 0x240 is l2MessagingBlocksOffsets.
//

assertEq(filteredAddressesLength, 1, "0x240 -> filteredAddresses: length should be 1");
assertEq(
    filteredAddressesFirstElement,
    address(0xDEAD),
    "0x240 -> filteredAddresses[0] should be 0xDEAD (NOT messaging offsets data)"
);

//
// PROOF 4: Dereferencing offset at 0x260 yields l2MessagingBlocksOffsets
// The NatSpec says this is at 0x240, but it's actually at 0x260.
//

assertEq(messagingOffsetsLength, 2, "0x260 -> l2MessagingBlocksOffsets: length should be 2
↳ bytes");

//
// SUMMARY:
// NatSpec claims: 0x220 = l2MerkleRoots, 0x240 = l2MessagingBlocksOffsets
// Reality: 0x220 = l2MerkleRoots, 0x240 = filteredAddresses, 0x260 =
↳ l2MessagingBlocksOffsets
//
// The NatSpec is missing filteredAddresses and has l2MessagingBlocksOffsets
// at the wrong offset.
//
}
}

```

Run with: `forge test --match-test test_finalizationDataV4_calldata_layout`

Linea: Fixed in commit [857c4b7](#).

Cyfrin: Verified.

7.3 Gas Optimization

7.3.1 Remove unused storage slots in new contracts

Description: Remove unused storage slots in new contracts:

```
AddressFilter.sol
13:  bool public useAddressFilter = true;
```

Linea: Fixed in commit [857c4b7](#).

Cyfrin: Verified.

7.3.2 More efficient usage of LibRLP in ForcedTransactionGateway::_buildAccessList, submitForcedTransaction

Description: A more efficient implementation of ForcedTransactionGateway::_buildAccessList avoids:

- empty initialization of acct (see this [comment](#))
- calling LibRLP.p() since it does [nothing](#)

```
function _buildAccessList(AccessList[] memory _accessList) internal pure returns (LibRLP.List memory
↳ list) {
    unchecked {
        // list is already zero-initialized (empty list)
        for (uint256 i; i < _accessList.length; ++i) {
            LibRLP.List memory keys; // Empty list, no p() needed
            bytes32[] memory ks = _accessList[i].storageKeys;
            for (uint256 j; j < ks.length; ++j) {
                bytes memory b = new bytes(32);
                assembly {
                    mstore(add(b, 0x20), mload(add(ks, add(0x20, shl(5, j)))))
                }
                keys = LibRLP.p(keys, b);
            }
            LibRLP.List memory acct = LibRLP.p(_accessList[i].contractAddress); // single-arg overload
            acct = LibRLP.p(acct, keys);
            list = LibRLP.p(list, acct);
        }
    }
}
```

The same techniques can be used in ForcedTransactionGateway::submitForcedTransaction:

```
-  LibRLP.List memory transactionFieldList = LibRLP.p();
-  transactionFieldList = LibRLP.p(transactionFieldList, DESTINATION_CHAIN_ID);
+  LibRLP.List memory transactionFieldList = LibRLP.p(DESTINATION_CHAIN_ID);
```

Linea: Fixed in commit [857c4b7](#).

Cyfrin: Verified.

7.3.3 Refactor to avoid two calls to AddressFilter::addressIsFiltered

Description: The intention is that the address filter will be enabled and hence every call to ForcedTransactionGateway::submitForcedTransaction will make two external calls to AddressFilter::addressIsFiltered:

```
if (useAddressFilter) {
    require(!ADDRESS_FILTER.addressIsFiltered(signer), AddressIsFiltered());
    require(!ADDRESS_FILTER.addressIsFiltered(_forcedTransaction.to), AddressIsFiltered());
}
```

The most common case will likely be that `signer == _forcedTransaction.to` which also results in duplicate identical storage reads. This can be refactored more efficiently by:

- 1) Creating a function `AddressFilter::addressesAreFiltered` which takes two inputs:

```
function addressesAreFiltered(address _addr1, address _addr2) external view returns (bool) {
    if (_addr1 == _addr2) {
        return filteredAddresses[_addr1];
    }
    return filteredAddresses[_addr1] || filteredAddresses[_addr2];
}
```

- 2) Calling this function once in `ForcedTransactionGateway::submitForcedTransaction`:

```
if (useAddressFilter)
    require(!ADDRESS_FILTER.addressesAreFiltered(signer, _forcedTransaction.to),
        ↪ AddressIsFiltered());
```

This solution results in only 1 external and optimizes away the duplicate identical storage read that would occur in the most common case where `signer == _forcedTransaction.to`.

Linea: Fixed in commit [857c4b7](#) by only making the second call if `signer != _forcedTransaction.to`.

Cyfrin: Verified.